

app\adc_ads124s08.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      adc_ads124s08.py
4  @brief     Driver pour l'ADC ADS124S08 avec communication SPI.
5  @details   Ce module fournit une interface complète pour la communication avec
6             l'ADC ADS124S08 via SPI en mode 1.0. Supporte la mesure de résistance
7             et température avec configuration automatique des registres.
8             Séquence: Reset → Configuration → Mesure bloquante avec DRDY.
9  @author    Léo Mendes
10 @project    2510_RegulationsChauffage
11 @Mandant    Oblo_solution
12 @date       2025-09-18
13 @version    1.0.0
14 @copyright  Copyright (c) 2025
15 """
16
17 from __future__ import annotations
18 # Bibliothèque pour la communication SPI
19 import spidev
20 # Module pour la gestion du temps et délais
21 import time
22 # Bibliothèque pour la gestion des GPIO
23 from periphery import GPIO
24 # Gestion des signaux système pour interruption propre
25 import signal
26 # Gestion des threads pour les opérations non-bloquantes
27 import threading
28 # Importation des constantes matérielles CMS
29 from pins_cm5 import SPI1_BUS, SPI1_DEV0, SPI_ADC_SPEED_HZ, GPIO_CHIP_PATH, ADC_DRDY
30 # Fonction de conversion résistance vers température
31 from app.temperature_conversion import resistance_to_temperature_dynamic
32 # Tables de conversion des différents capteurs
33 from app.sensor_profiles import SENSOR_TABLES
34
35 # -----
36 # Variables globales pour la gestion d'interruption
37 # -----
38
39 # Flag global pour indiquer une demande d'arrêt
40 _stop_requested = False
41 # Lock pour la synchronisation thread-safe
42 _stop_lock = threading.Lock()
43
44 def request_stop():
45     """
46     @brief   Demande l'arrêt global du programme.
47     @details Active le flag global d'arrêt de manière thread-safe.
48     """
49     global _stop_requested
50     with _stop_lock:
51         _stop_requested = True
52         print("[ADC] Arrêt demandé - interruption des opérations en cours...")
53
54 def is_stop_requested():
55     """
56     @brief   Vérifie si un arrêt a été demandé.
57     @details Retourne l'état du flag d'arrêt de manière thread-safe.
58     @return  True si arrêt demandé, False sinon.
59     """
60     global _stop_requested
61     with _stop_lock:
62         return _stop_requested
63
64 def reset_stop_flag():
65     """
66     @brief   Remet à zéro le flag d'arrêt.
67     @details Permet de redémarrer les opérations après un arrêt.
68     """
69     global _stop_requested
70     with _stop_lock:
71         _stop_requested = False
72
73 # Gestionnaire de signal pour Ctrl+C
74 def signal_handler(signum, frame):
75     """
76     @brief   Gestionnaire pour les signaux système (Ctrl+C).
77     @details Capture SIGINT et SIGTERM pour arrêter proprement le programme.
78     """
79     print(f"\n[Signal] Signal {signum} reçu - arrêt en cours...")
80     request_stop()
81
82 # Installation du gestionnaire de signal
83 signal.signal(signal.SIGINT, signal_handler)
84 signal.signal(signal.SIGTERM, signal_handler)

```

```

85
86
87 # -----
88 # Constantes des registres ADC ADS124S08
89 # -----
90
91 # Registre d'identification de l'ADC
92 REG_ID      = 0x00
93 # Registre de statut et flags
94 REG_STATUS  = 0x01
95 # Registre de configuration du multiplexeur d'entrée
96 REG_INPMUX  = 0x02
97 # Registre de configuration de l'amplificateur programmable
98 REG_PGA     = 0x03
99 # Registre de configuration du taux d'échantillonnage
100 REG_DATARATE = 0x04
101 # Registre de configuration de la référence de tension
102 REG_REF     = 0x05
103 # Registre de configuration du multiplexeur IDAC
104 REG_IDACMUX  = 0x07
105 # Registre de configuration système générale
106 REG_SYS     = 0x09
107 # Registre de configuration de la magnitude IDAC
108 REG_IDACMAG  = 0x06
109
110
111 # -----
112 # Constantes des commandes ADC ADS124S08
113 # -----
114
115 # Commande de réinitialisation complète de l'ADC
116 CMD_RESET   = 0x06
117 # Commande de démarrage d'une conversion
118 CMD_START   = 0x08
119 # Commande d'arrêt de conversion
120 CMD_STOP    = 0x0A
121 # Commande de lecture des données de conversion
122 CMD_RDATA   = 0x12
123
124
125 # -----
126 # Constantes de calcul et mapping
127 # -----
128
129 # Valeur maximale pour un code 24 bits signé (pleine échelle)
130 FS = (1 << 23) - 1
131
132 # Mapping des canaux physiques vers les entrées analogiques de l'ADC
133 CHANNELS = {
134     # Canal 1: entrée positive AIN1, négative AIN2
135     1: {"ainp_idx": 1, "ainn_idx": 2},
136     # Canal 2: entrée positive AIN4, négative AIN5
137     2: {"ainp_idx": 4, "ainn_idx": 5},
138     # Canal 3: entrée positive AIN7, négative AIN8
139     3: {"ainp_idx": 7, "ainn_idx": 8},
140     # Canal 4: entrée positive AIN10, négative AIN11
141     4: {"ainp_idx": 10, "ainn_idx": 11}
142 }
143
144
145 # -----
146 # Fonctions d'encodage des paramètres ADC
147 # -----
148
149 def encode_gain(pga_gain):
150     """
151     @brief      Encode le gain PGA en code binaire pour le registre.
152     @details    Convertit une valeur de gain décimale en code binaire 3 bits
153                 pour la configuration du registre PGA de l'ADS124S08.
154
155     @param pga_gain Valeur du gain PGA (1, 2, 4, 8, 16, 32, 64, 128).
156
157     @return     Code binaire 3 bits correspondant au gain.
158
159     @exception  ValueError si le gain fourni n'est pas supporté.
160     @see        configure_channel
161     """
162     # Gain unitaire (0 dB)
163     if pga_gain == 1:
164         return 0
165     # Gain x2 (6 dB)
166     if pga_gain == 2:
167         return 1
168     # Gain x4 (12 dB)
169     if pga_gain == 4:
170         return 2
171     # Gain x8 (18 dB)

```

```

172     if pga_gain == 8:
173         return 3
174     # Gain x16 (24 dB)
175     if pga_gain == 16:
176         return 4
177     # Gain x32 (30 dB)
178     if pga_gain == 32:
179         return 5
180     # Gain x64 (36 dB)
181     if pga_gain == 64:
182         return 6
183     # Gain x128 (42 dB)
184     if pga_gain == 128:
185         return 7
186     # Erreur si gain non supporté
187     raise ValueError("PGA gain invalide: " + str(pga_gain))
188
189
190 def encode_idac_uA(idac_uA):
191     """
192     @brief      Encode la magnitude IDAC en code binaire pour le registre.
193     @details    Convertit une valeur de courant IDAC en microampères en code
194                 binaire 4 bits pour la configuration du registre IDACMAG.
195
196     @param idac_uA    Courant IDAC en microampères (10, 50, 100, 250, 500, 1000, 1500, 2000).
197
198     @return        Code binaire 4 bits correspondant au courant.
199
200     @exception    ValueError si le courant fourni n'est pas supporté.
201     @see          configure_channel
202     """
203     # Courant IDAC de 10 µA
204     if idac_uA == 10:
205         return 1
206     # Courant IDAC de 50 µA
207     if idac_uA == 50:
208         return 2
209     # Courant IDAC de 100 µA
210     if idac_uA == 100:
211         return 3
212     # Courant IDAC de 250 µA
213     if idac_uA == 250:
214         return 4
215     # Courant IDAC de 500 µA
216     if idac_uA == 500:
217         return 5
218     # Courant IDAC de 1000 µA (1 mA)
219     if idac_uA == 1000:
220         return 6
221     # Courant IDAC de 1500 µA (1.5 mA)
222     if idac_uA == 1500:
223         return 7
224     # Courant IDAC de 2000 µA (2 mA)
225     if idac_uA == 2000:
226         return 8
227     # Erreur si courant non supporté
228     raise ValueError("IDAC µA non supporté: " + str(idac_uA))
229
230
231 def sign_extend_24(b0, b1, b2):
232     """
233     @brief      Étend le signe d'un code ADC 24 bits vers 32 bits.
234     @details    Convertit un code ADC 24 bits signé en entier Python 32 bits
235                 en préservant le signe selon la représentation en complément à 2.
236
237     @param b0    Octet de poids fort (MSB) du code 24 bits.
238     @param b1    Octet du milieu du code 24 bits.
239     @param b2    Octet de poids faible (LSB) du code 24 bits.
240
241     @return        Valeur entière signée 32 bits correspondante.
242
243     @note        Nécessaire car Python gère automatiquement les entiers de taille variable.
244     @see          read_code24
245     """
246     # Reconstruction du mot 24 bits
247     raw = (b0 << 16) | (b1 << 8) | b2
248     # Test du bit de signe (bit 23)
249     if (raw & 0x800000) != 0:
250         # Extension du signe vers 32 bits
251         value = raw | 0xFF000000
252         # Conversion en entier signé Python
253         value = value - (1 << 32)
254         return value
255     # Valeur positive, pas d'extension nécessaire
256     return raw
257
258

```

```

259 # -----
260 # Classe principale du driver ADC
261 # -----
262
263 class Ads124s08:
264     """
265     @brief      Driver pour l'ADC ADS124S08 avec interface SPI.
266     @details    Cette classe encapsule toutes les fonctionnalités nécessaires
267                 pour communiquer avec l'ADC ADS124S08 via SPI en mode 1.0.
268                 Supporte la configuration automatique, la mesure de résistance
269                 et la conversion en température.
270
271     @note       L'ADC doit être connecté via SPI1 et le signal DRDY sur GPIO.
272     @see        configure_channel, measure_resistance, measure_temperature
273     """
274
275     def __init__(self):
276         """
277         @brief      Constructeur de la classe Ads124s08.
278         @details    Initialise la communication SPI, configure le GPIO DRDY,
279                 effectue un reset de l'ADC et vérifie la communication
280                 en lisant le registre d'identification.
281
282         @exception  Peut lever des exceptions SPI ou GPIO en cas d'échec.
283         @post       L'ADC est prêt pour la configuration et les mesures.
284         """
285         # Initialisation de l'interface SPI avec les paramètres du CM5
286         # Création de l'objet SPI
287         self.spi = spidev.SpiDev()
288         # Ouverture du bus SPI1, device 0
289         self.spi.open(SPI1_BUS, SPI1_DEV0)
290         # Configuration en mode SPI 1.0 (CPOL=0, CPHA=1)
291         self.spi.mode = 1
292         # Configuration de la vitesse SPI selon CM5
293         self.spi.max_speed_hz = SPI_ADC_SPEED_HZ
294         # Configuration de la taille des mots SPI à 8 bits
295         self.spi.bits_per_word = 8
296
297         # Initialisation du GPIO pour le signal DRDY (Data Ready)
298         # Configuration du GPIO DRDY en entrée
299         self.gpio_drdy = GPIO(GPIO_CHIP_PATH, ADC_DRDY, "in")
300
301         # Séquence de réinitialisation de l'ADC
302         # Envoi de la commande RESET via SPI
303         self.spi.xfer2([CMD_RESET])
304         # Délai d'attente post-reset (2 ms minimum)
305         time.sleep(0.002)
306
307         # Purge des flags de statut après reset
308         # Effacement des flags du registre STATUS
309         self._wreg(REG_STATUS, [0x00])
310
311         # Vérification de la communication avec l'ADC via lecture du registre ID
312         # Lecture du registre d'identification
313         adc_id = self.read_id()
314         # Test de validité de la réponse
315         if adc_id is None:
316             # Erreur de communication
317             print("[ADC] Erreur: aucune réponse sur le registre ID (0x00)")
318         else:
319             # Affichage de l'ID pour vérification
320             print("[ADC] ID (0x00) = 0x" + format(adc_id, "02X"))
321
322     def read_id(self):
323         """
324         @brief      Lit le registre d'identification de l'ADC.
325         @details    Effectue une lecture du registre ID (0x00) pour vérifier
326                 la communication SPI et identifier le type d'ADC connecté.
327
328         @return     Valeur du registre ID (0x00-0xFF) ou None en cas d'erreur.
329
330         @note       Utilisé principalement pour la vérification de communication.
331         @see        __init__
332         """
333         # Commande RREG pour registre 0x00, 1 byte
334         rx = self.spi.xfer2([0x20, 0x00, 0x00])
335         # Vérification de la longueur de réponse
336         if len(rx) >= 3:
337             # Retour de la valeur du registre ID
338             return rx[2]
339         # Erreur de communication SPI
340         return None
341
342     def close(self):
343         """
344         @brief      Ferme les interfaces SPI et GPIO proprement.
345         @details    Libère les ressources système utilisées par le driver.

```

```

346             Méthode à appeler avant la destruction de l'objet.
347
348     @return      Rien.
349
350     @note        Ignore les erreurs de fermeture pour éviter les exceptions.
351     @see         __init__
352     """
353     try:
354         # Fermeture de l'interface SPI
355         self.spi.close()
356     # Ignore les erreurs de fermeture SPI
357     except Exception:
358         pass
359     try:
360         # Fermeture de l'interface GPIO DRDY
361         self.gpio_drdy.close()
362     # Ignore les erreurs de fermeture GPIO
363     except Exception:
364         pass
365
366 def _rreg(self, addr, nbytes):
367     """
368     @brief        Lit un ou plusieurs registres de l'ADC via SPI.
369     @details      Utilise la commande RREG de l'ADS124S08 pour lire
370                   des registres consécutifs starting à l'adresse donnée.
371
372     @param addr   Adresse du premier registre à lire (0x00-0x1F).
373     @param nbytes Nombre d'octets à lire.
374
375     @return       Liste des valeurs lues depuis les registres.
376
377     @note         Méthode privée pour les opérations internes du driver.
378     @see          _wreg
379     """
380     # Construction de la commande RREG
381     cmd = 0x20 | (addr & 0x1F)
382     # Calcul du compteur (nbytes - 1)
383     count = nbytes - 1
384     # Initialisation du buffer de transmission
385     tx = [cmd, count]
386     # Index de boucle pour les octets dummy
387     index = 0
388     # Ajout des octets dummy pour la lecture
389     while index < nbytes:
390         # Octet dummy pour chaque byte à lire
391         tx.append(0x00)
392         # Incréméntation de l'index
393         index = index + 1
394     # Transmission SPI bidirectionnelle
395     rx = self.spi.xfer2(tx)
396     # Extraction des données (après cmd et count)
397     data = rx[2:]
398     # Retour des données lues
399     return data
400
401 def _wreg(self, addr, data_bytes):
402     """
403     @brief        Écrit dans un ou plusieurs registres de l'ADC via SPI.
404     @details      Utilise la commande WREG de l'ADS124S08 pour écrire
405                   des données dans des registres consécutifs.
406
407     @param addr   Adresse du premier registre à écrire (0x00-0x1F).
408     @param data_bytes Liste des octets à écrire dans les registres.
409
410     @return       Rien.
411
412     @note         Méthode privée pour les opérations internes du driver.
413     @see          _rreg
414     """
415     # Construction de la commande WREG
416     cmd = 0x40 | (addr & 0x1F)
417     # Calcul du compteur (nbytes - 1)
418     count = len(data_bytes) - 1
419     # Initialisation du buffer de transmission
420     tx = [cmd, count]
421     # Ajout des données à transmettre
422     tx = tx + list(data_bytes)
423     # Transmission SPI (pas de données retour)
424     self.spi.xfer2(tx)
425
426 def _sclk_nudge(self):
427     """
428     @brief        Envoie un coup de pouce sur SCLK pour synchroniser DRDY.
429     @details      Effectue une lecture factice du registre STATUS pour générer
430                   des transitions sur SCLK et forcer DRDY à l'état HIGH.
431
432     @return       Rien.

```

```

433
434 @note          Utilisé quand DRDY reste bloqué à LOW de manière inattendue.
435 @see           _ensure_drdy_high
436 """
437 # Lecture factice pour générer des transitions SCLK
438 _ = self._rreg(REG_STATUS, 1)
439 # Délai court pour la stabilisation
440 time.sleep(0.0001)
441
442 def _ensure_drdy_high(self, timeout_ms):
443     """
444     @brief       Assure que le signal DRDY est à l'état HIGH.
445     @details     Attend que DRDY passe à HIGH avec possibilité de coup de pouce
446                 sur SCLK si nécessaire. Timeout configurable.
447
448     @param timeout_ms  Timeout en millisecondes pour l'attente.
449
450     @return        True si DRDY est HIGH, False en cas de timeout.
451
452     @note          Méthode critique pour la synchronisation des conversions.
453     @see           wait_drdy_falling_edge
454     """
455     # Timestamp de début d'attente
456     t0 = time.time()
457     # Flag pour éviter les multiples coups de pouce
458     kicked = False
459     # Boucle d'attente infinie avec timeout
460     while True:
461         # Lecture de l'état actuel de DRDY
462         val = self.gpio_drdy.read()
463         # Test si DRDY est passé à HIGH
464         if val is True:
465             # Succès: DRDY est HIGH
466             return True
467         # Premier passage avec DRDY LOW
468         if kicked is False:
469             # Coup de pouce sur SCLK
470             self._sclk_nudge()
471             # Marquer le coup de pouce comme effectué
472             kicked = True
473         # Test du timeout en secondes
474         if time.time() - t0 > float(timeout_ms) / 1000.0:
475             # Échec: timeout atteint
476             return False
477         # Délai court entre les lectures
478         time.sleep(0.001)
479
480 def wait_drdy_falling_edge(self, timeout_s):
481     """
482     @brief       Attend un front descendant sur DRDY (HIGH -> LOW) avec timeout.
483     @details     Surveille le signal DRDY pour détecter une transition de HIGH
484                 vers LOW, indiquant qu'une nouvelle conversion est disponible.
485                 Peut être interrompu par Ctrl+C ou signal d'arrêt.
486
487     @param timeout_s  Timeout en secondes pour l'attente du front.
488
489     @return        True si front descendant détecté, False en cas de timeout ou interruption.
490
491     @note          Méthode cruciale pour la synchronisation des mesures.
492     @see           measure_resistance
493     """
494     # Timestamp de début d'attente
495     t0 = time.time()
496     # État initial de DRDY
497     prev = self.gpio_drdy.read()
498
499     # Affichage de l'état initial pour diagnostic
500     print(f"[ADC] Attente front descendant DRDY, état initial: {prev}, timeout: {timeout_s}s")
501     print("[ADC] Appuyez sur Ctrl+C pour interrompre l'attente...")
502
503     # Compteur pour les messages de debug périodiques
504     debug_counter = 0
505
506     # Boucle d'attente infinie avec timeout
507     while True:
508         # Vérification du flag d'arrêt global
509         if is_stop_requested():
510             print("[ADC] Interruption demandée - arrêt de l'attente DRDY")
511             return False
512
513         # Lecture de l'état actuel de DRDY
514         val = self.gpio_drdy.read()
515
516         # Affichage périodique pour diagnostic (toutes les 500 itérations)
517         debug_counter += 1
518         if debug_counter % 500 == 0:
519             elapsed = time.time() - t0

```

```

520         print(f"[ADC] Attente DRDY... État: {val}, Temps écoulé: {elapsed:.1f}s (Ctrl+C pour arrêter)")
521
522     # Détection du front descendant
523     if prev is True and val is False:
524         # Message de confirmation
525         print("[ADC] Front descendant DRDY détecté")
526         # Succès: front descendant détecté
527         return True
528     # Mémorisation de l'état précédent
529     prev = val
530     # Test du timeout
531     if time.time() - t0 > float(timeout_s):
532         # Message d'erreur timeout avec état final
533         print(f"[ADC] Timeout DRDY front descendant ! État final: {val}")
534         print("[ADC] Vérifiez: connexion DRDY, configuration ADC, horloge SPI")
535         # Échec: timeout atteint
536         return False
537     # Délai court entre les lectures
538     time.sleep(0.001)
539
540 def set_single_shot_lowlatency(self, dr_nibble):
541     """
542     @brief      Configure le mode single-shot low-latency de l'ADC.
543     @details    Configure le registre DATARATE pour utiliser le mode
544                 single-shot avec filtre low-latency et data rate spécifié.
545
546     @param dr_nibble  Code de data rate (4 bits, 0x00-0x0F).
547
548     @return        Rien.
549
550     @note          Mode optimisé pour des mesures rapides et ponctuelles.
551     @see           configure_channel
552     """
553     # Initialisation de la valeur du registre
554     value = 0
555     # MODE=1 pour single-shot
556     value = value | (1 << 5)
557     # FILTER=1 pour low-latency
558     value = value | (1 << 4)
559     # DR=data rate (4 bits bas)
560     value = value | (dr_nibble & 0x0F)
561     # Écriture dans le registre DATARATE
562     self._wreg(REG_DATARATE, [value])
563
564 def configure_channel(self, channel_index, pga_gain, idac_uA):
565     """
566     @brief      Configure un canal ADC avec ses paramètres de mesure.
567     @details    Configure complètement un canal ADC avec le gain PGA,
568                 le courant IDAC et le routage des entrées analogiques.
569                 Effectue une configuration séquentielle de tous les registres.
570
571     @param channel_index  Index du canal physique (1-4).
572     @param pga_gain       Gain de l'amplificateur programmable (1-128).
573     @param idac_uA        Courant IDAC en microampères (10-2000).
574
575     @return        Rien.
576
577     @exception      ValueError si le canal ou les paramètres sont invalides.
578     @see           encode_gain, encode_idac_uA
579     """
580     # Validation du canal demandé
581     # Test de validité du canal
582     if channel_index not in CHANNELS:
583         # Exception si canal invalide
584         raise ValueError("Canal ADC inconnu: " + str(channel_index))
585     # Récupération des paramètres du canal
586     ch = CHANNELS[channel_index]
587     # Index de l'entrée positive
588     ainp = ch["ainp_idx"]
589     # Index de l'entrée négative
590     ainm = ch["ainm_idx"]
591     print("[ADC] Configuration canal " + str(channel_index) + " gain=" + str(pga_gain) + " IDAC=" + str(idac_uA) + "uA")
592
593     # Configuration du multiplexeur d'entrée (INPMUX)
594     # Construction de la valeur INPMUX
595     inpmux_val = ((ainp & 0x0F) << 4) | (ainm & 0x0F)
596     # Affichage pour debug
597     print("[ADC] INPMUX=0x" + format(inpmux_val, "02X"))
598     # Écriture dans le registre INPMUX
599     self._wreg(REG_INPMUX, [inpmux_val])
600
601     # Configuration de l'amplificateur programmable (PGA)
602     # Encodage du gain en code binaire
603     gain_code = encode_gain(pga_gain)
604     # Initialisation de la valeur PGA
605     pga_val = 0
606     # PGA_EN = 1 (activation du PGA)

```

```

607         pga_val = pga_val | (1 << 3)
608         # GAIN = code du gain (3 bits)
609         pga_val = pga_val | (gain_code & 0x07)
610         # Affichage pour debug
611         print("[ADC] PGA=0x" + format(pga_val, "02X"))
612         # Écriture dans le registre PGA
613         self._wreg(REG_PGA, [pga_val])
614         # Lecture de vérification
615         val = self._rreg(REG_PGA, 1)
616         # Affichage de la valeur lue
617         print("[ADC DEBUG] PGA readback: 0x" + format(val[0], "02X"))
618
619         # Configuration du mode de conversion (single-shot low-latency)
620         # DR=0x04 pour data rate optimisé
621         self.set_single_shot_lowlatency(0x04)
622
623         # Configuration de la référence de tension (REF)
624         # REFSEL=00 (REFP0/REFN0), REFCON=10 (ref interne ON)
625         print("[ADC] REF=0x12")
626         # Écriture dans le registre REF
627         self._wreg(REG_REF, [0x12])
628         # Attente pour stabilisation de la référence interne
629         time.sleep(0.006)
630         # Lecture de vérification
631         val = self._rreg(REG_REF, 1)
632         # Affichage de la valeur lue
633         print("[ADC DEBUG] REF readback: 0x" + format(val[0], "02X"))
634
635         # Configuration de la magnitude IDAC
636         # Encodage du courant IDAC
637         mag_code = encode_idac_uA(idac_uA)
638         # Affichage pour debug
639         print("[ADC] IDACMAG=0x" + format(mag_code & 0x0F, "02X"))
640         # Écriture dans le registre IDACMAG
641         self._wreg(REG_IDACMAG, [mag_code & 0x0F])
642         # Lecture de vérification
643         val = self._rreg(REG_IDACMAG, 1)
644         # Affichage de la valeur lue
645         print("[ADC DEBUG] IDACMAG readback: 0x" + format(val[0], "02X"))
646
647         # Configuration du routage IDAC (IDACMUX)
648         # Calcul du routage IDAC1: 0, 3, 6, 9
649         idac1_route = 0 + (channel_index - 1) * 3
650         # Limitation basse du routage
651         if idac1_route < 0:
652             idac1_route = 0
653         # Limitation haute du routage
654         if idac1_route > 15:
655             idac1_route = 15
656         # IDAC2 déconnecté (0x0F)
657         idac2_route = 0x0F
658         # Construction de la valeur IDACMUX
659         idacmux_val = ((idac2_route & 0x0F) << 4) | (idac1_route & 0x0F)
660         # Affichage pour debug
661         print("[ADC] IDACMUX=0x" + format(idacmux_val, "02X"))
662         # Écriture dans le registre IDACMUX
663         self._wreg(REG_IDACMUX, [idacmux_val])
664         # Lecture de vérification
665         val = self._rreg(REG_IDACMUX, 1)
666         # Affichage de la valeur lue
667         print("[ADC DEBUG] IDACMUX readback: 0x" + format(val[0], "02X"))
668
669         # Délai de stabilisation après configuration complète
670         # Attente pour stabilisation des paramètres
671         time.sleep(0.001)
672
673         # Lecture de tous les registres clés pour debug final
674         reg_map = {
675             "INPMUX": REG_INPMUX,
676             "PGA": REG_PGA,
677             "DATARATE": REG_DATARATE,
678             "REF": REG_REF,
679             "IDACMAG": REG_IDACMAG,
680             "IDACMUX": REG_IDACMUX
681         }
682         # Parcours de tous les registres importants
683         for name, addr in reg_map.items():
684             # Lecture de chaque registre
685             val = self._rreg(addr, 1)
686             # Affichage de l'état final
687             print(f"[ADC] {name} {0x{addr:02X}} = 0x{val[0]:02X}")
688
689     def start(self):
690         """
691         @brief      Démarre une conversion ADC.
692         @details    Assure que DRDY est à HIGH puis envoie la commande START
693                     pour initier une nouvelle conversion sur le canal configuré.

```



```

694         Peut être interrompu par Ctrl+C ou signal d'arrêt.
695
696     @return      True si START envoyé avec succès, False en cas d'erreur ou interruption.
697
698     @note        DRDY doit être HIGH avant le START pour une conversion valide.
699     @see         stop, wait_drdy_falling_edge
700     """
701     # Attente que DRDY remonte à HIGH avant le démarrage
702     # Timestamp de début d'attente
703     t0 = time.time()
704     print("[ADC] Attente que DRDY soit HIGH avant START... (Ctrl+C pour arrêter)")
705
706     # Boucle tant que DRDY n'est pas HIGH
707     while self.gpio_drdy.read() is not True:
708         # Vérification du flag d'arrêt global
709         if is_stop_requested():
710             print("[ADC] Interruption demandée - arrêt de l'attente DRDY HIGH")
711             return False
712
713         # Lecture du registre STATUS pour kick SCLK
714         self._rreg(REG_STATUS, 1)
715         # Délai court entre les tentatives
716         time.sleep(0.001)
717         # Timeout de 5 secondes (augmenté pour diagnostic)
718         if time.time() - t0 > 5.0:
719             # Message d'erreur détaillé
720             print("[ADC] ERREUR: DRDY n'est pas remonté HIGH avant START !")
721             print("[ADC] Vérifiez la connexion DRDY et l'alimentation ADC")
722             # Retour d'erreur sans envoyer START
723             return False
724
725     # Vérification finale du flag d'arrêt avant d'envoyer START
726     if is_stop_requested():
727         print("[ADC] Interruption demandée - annulation de START")
728         return False
729
730     # DRDY est maintenant HIGH, on peut envoyer START
731     print("[ADC] DRDY HIGH détecté, envoi de la commande START")
732     # Envoi de la commande START pour débiter la conversion
733     # Transmission de la commande START via SPI
734     self.spi.xfer2([CMD_START])
735     return True
736
737 def stop(self):
738     """
739     @brief        Arrête une conversion ADC en cours.
740     @details      Envoie la commande STOP pour interrompre toute conversion
741                   en cours et remettre l'ADC en état idle.
742
743     @return       Rien.
744
745     @note         Utilisé après lecture des données ou en cas d'erreur.
746     @see          start, read_code24
747     """
748     # Transmission de la commande STOP via SPI
749     self.spi.xfer2([CMD_STOP])
750
751 def read_code24(self):
752     """
753     @brief        Lit un code de données 24 bits depuis l'ADC.
754     @details      Utilise la commande RDATA pour lire les 3 octets de données
755                   de conversion et les convertit en entier signé.
756
757     @return       Valeur signée 24 bits ou None en cas d'erreur SPI.
758
759     @note         Doit être appelé après détection du front descendant DRDY.
760     @see          sign_extend_24, measure_resistance
761     """
762     # Commande RDATA + 3 octets dummy
763     rx = self.spi.xfer2([CMD_RDATA, 0x00, 0x00, 0x00])
764     # Vérification de la longueur de réponse
765     if len(rx) < 4:
766         # Message d'erreur
767         print("[ADC] Erreur: réponse SPI trop courte " + str(rx))
768         # Retour d'erreur
769         return None
770     # Octet de poids fort (MSB)
771     b0 = rx[1]
772     # Octet du milieu
773     b1 = rx[2]
774     # Octet de poids faible (LSB)
775     b2 = rx[3]
776     # Extension du signe vers 32 bits
777     value = sign_extend_24(b0, b1, b2)
778     # Retour de la valeur signée
779     return value
780

```

```

781 def measure_resistance(self, rref_ohm, pga_gain, timeout_s):
782     """
783     @brief      Mesure la résistance d'une sonde via l'ADC.
784     @details    Effectue une mesure complète: démarrage de conversion,
785                 attente DRDY, lecture du code et calcul de la résistance
786                 selon la formule ratiométrique.
787
788     @param rref_ohm  Résistance de référence en ohms.
789     @param pga_gain  Gain PGA utilisé pour la mesure.
790     @param timeout_s Timeout en secondes pour la conversion.
791
792     @return        Résistance mesurée en ohms ou None en cas d'erreur.
793
794     @note          Utilise la formule:  $R = |code| / FS * (Rref / gain)$ .
795     @see           measure_temperature, configure_channel
796     """
797     print("[ADC] Mesure résistance: rref=" + str(rref_ohm) + " gain=" + str(pga_gain) + " timeout=" + str(timeout_s))
798
799     # Démarrage de la conversion ADC
800     # Envoi de la commande START
801     start_success = self.start()
802     if not start_success:
803         print("[ADC] Erreur: Impossible de démarrer la conversion, DRDY non disponible")
804         return None
805
806     # Attente du signal DRDY indiquant la fin de conversion
807     # Attente du front descendant DRDY
808     ok = self.wait_drdy_falling_edge(timeout_s)
809     # Test du résultat de l'attente
810     if ok is False:
811         # Arrêt de la conversion en cas de timeout
812         self.stop()
813         # Message d'erreur
814         print("[ADC] Erreur: DRDY non détecté, mesure annulée")
815         # Retour d'erreur
816         return None
817
818     # Lecture du code de conversion 24 bits
819     # Lecture des données de conversion
820     code = self.read_code24()
821     # Arrêt de la conversion
822     self.stop()
823
824     # Validation du code lu
825     # Test de validité du code
826     if code is None:
827         # Message d'erreur
828         print("[ADC] Erreur: code ADC non lu")
829         # Retour d'erreur
830         return None
831
832     # Affichage des paramètres de debug
833     # Affichage du code brut
834     print(f"[ADC DEBUG] Code brut: {code}")
835     # Affichage de la pleine échelle
836     print(f"[ADC DEBUG] FS: {FS}")
837     # Affichage des paramètres de calcul
838     print(f"[ADC DEBUG] Rref: {rref_ohm} / Gain: {pga_gain}")
839
840     # Prise de la valeur absolue du code pour le calcul
841     # Test du signe du code
842     if code < 0:
843         # Prise de la valeur absolue
844         code = -code
845
846     # Calcul de la résistance selon la formule ratiométrique
847     # Calcul du ratio code/pleine échelle
848     ratio = float(code) / float(FS)
849     # Calcul de la résistance de la sonde
850     r_sonde = ratio * (float(rref_ohm) / float(pga_gain))
851
852     # Affichage du résultat
853     print(f"[ADC] Résistance mesurée: {r_sonde:.1f} ohms")
854     # Retour de la résistance mesurée
855     return r_sonde
856
857 def measure_temperature(self, sensor_name, rref_ohm, pga_gain, timeout_s):
858     """
859     @brief      Mesure la température d'un capteur via l'ADC.
860     @details    Effectue une mesure de résistance puis utilise les tables
861                 de conversion de capteurs pour obtenir la température
862                 correspondante en degrés Celsius.
863
864     @param sensor_name  Nom du capteur (doit exister dans SENSOR_TABLES).
865     @param rref_ohm     Résistance de référence en ohms.
866     @param pga_gain     Gain PGA utilisé pour la mesure.
867     @param timeout_s    Timeout en secondes pour la conversion.

```

```

868
869         @return          Température mesurée en °C ou None si erreur.
870
871     @exception          Retourne None si capteur inconnu ou hors plage.
872     @see               mesure_resistance, resistance_to_temperature_dynamic
873     """
874     # Mesure de la résistance du capteur
875     # Appel de la mesure de résistance
876     resistance = self.mesure_resistance(rref_ohm, pga_gain, timeout_s)
877     # Test de validité de la mesure
878     if resistance is None:
879         # Message d'erreur
880         print("[ADC] Erreur: Résistance non mesurée ou saturation détectée.")
881         # Retour d'erreur
882         return None
883
884     # Vérification de l'existence de la table de conversion
885     # Test de présence du capteur
886     if sensor_name not in SENSOR_TABLES:
887         # Message d'erreur
888         print(f"[ADC] Erreur: Table de conversion introuvable pour le capteur {sensor_name}.")
889         # Retour d'erreur
890         return None
891
892     # Conversion de la résistance en température
893     # Appel de la conversion
894     temperature = resistance_to_temperature_dynamic(resistance, sensor_name)
895     # Test de validité de la conversion
896     if temperature is None:
897         # Message d'erreur
898         print("[ADC] Erreur: Température non calculable (hors plage de la table).")
899         # Retour d'erreur
900         return None
901
902     # Affichage du résultat
903     print(f"[ADC] Température mesurée: {temperature:.2f} °C")
904     # Retour de la température mesurée
905     return temperature
906
907 def read_gain(self):
908     """
909     @brief          Lit le registre de gain PGA pour diagnostic.
910     @details        Effectue une lecture du registre PGA et affiche sa valeur
911                     pour vérification de la configuration du gain.
912
913     @return         Valeur brute du registre PGA (0x00-0xFF).
914
915     @note           Utilisé principalement pour le debug et la vérification.
916     @see            configure_channel
917     """
918     # Lecture du registre PGA
919     val = self._rreg(REG_PGA, 1)
920     # Affichage de la valeur lue
921     print(f"[ADC DEBUG] PGA Gain Register: 0x{val[0]:02X}")
922     # Retour de la valeur du registre
923     return val[0]
924
925 def read_ref(self):
926     """
927     @brief          Lit le registre de configuration de la référence.
928     @details        Effectue une lecture du registre REF et affiche sa valeur
929                     pour vérification de la configuration de la référence.
930
931     @return         Valeur brute du registre REF (0x00-0xFF).
932
933     @note           Utilisé principalement pour le debug et la vérification.
934     @see            configure_channel
935     """
936     # Lecture du registre REF
937     val = self._rreg(REG_REF, 1)
938     # Affichage de la valeur lue
939     print(f"[ADC DEBUG] Reference Register: 0x{val[0]:02X}")
940     # Retour de la valeur du registre
941     return val[0]
942
943 def read_inpmux(self):
944     """
945     @brief          Lit le registre de configuration du multiplexeur d'entrée.
946     @details        Effectue une lecture du registre INPMUX et affiche sa valeur
947                     pour vérification du routage des entrées analogiques.
948
949     @return         Valeur brute du registre INPMUX (0x00-0xFF).
950
951     @note           Utilisé principalement pour le debug et la vérification.
952     @see            configure_channel
953     """
954     # Lecture du registre INPMUX

```

```
955     val = self._rreg(REG_INPMUX, 1)
956     # Affichage de la valeur lue
957     print(f"[ADC DEBUG] INPMUX Register: 0x{val[0]:02X}")
958     # Retour de la valeur du registre
959     return val[0]
```